Report LINFO1361: Assignment 1

Group N°99

Student1: Siénou Lamien

Student2: Emile Ollier

February 28, 2024

1 Python AIMA (3 pts)

1. In order to perform a search, what are the classes that you must define or extend? Explain precisely why and where they are used inside a *tree_search*. Be concise! (e.g. do not discuss unchanged classes). **(1 pt)**

We must extend the Problem class. They are used inside a tree search to access the initial state and "work" its way throught its successor.

2. Both *breadth_first_graph_search* and *depth_first_graph_search* are making a call to the same function. How is their fundamental difference implemented (be explicit)? **(0.5 pt)**

They both call a pop function. In the case of Bfgs we pop from a Deque while for a Dfgs we pop from a list(behaving like a stack). Since we make the call on two different structure we call two different implementation.

3. What is the difference between the implementation of the *graph_search* and the *tree_search* methods and how does it impact the search methods? **(0.5 pt)**

While the graph search methods keep track of the visited states, the tree search methods does not. It impact the search method because keeping track of the visited set is great in case of a problem having cycles. This mean the graph search method will be more efficient in such cases. We can see the implementation differ by the addition of a explored structure in the code of the graph search methods.

4. What kind of structure is used to implement the *reached nodes minus the frontier list*? What properties must thus have the elements that you can put inside the reached nodes minus the frontier list? **(0.5 pt)**

It is a set and a dictionnary, the elements must not be duplicate as it would mean we visited twice a same state.

5. How technically can you use the implementation of the reached nodes minus the frontier list to deal with symmetrical states? (hint: if two symmetrical states are considered by the algorithm to be the same, they will not be visited twice) **(0.5 pt)**

2 The PacMan Problem (17 pts)

(a) **Describe** the set of possible actions your agent will consider at each state. Evaluate the branching factor**(1 pt)**

At each state the agent whill consider the following actions : "UP","DOWN","RIGHT","LEFT" which mean that the branching factor has an upperbound of 4.

(b) How would you build the action to avoid the walls? **(1 pt)**

Add a condition to check whether my next move goes into the wall or not. If the condition is true, I don't add this move to the list of actions to execute and check the other directions in the same way. If the condition is false, I continue in the same direction.

2. **Problem analysis.**

   (a) Explain the advantages and weaknesses of the following search strategies **on this problem** (not in general): depth first, breadth first. Which approach would you choose? **(2 pts)**

Their weaknesses and strength depends on where the fruit is. If the fruit is far away from our starting point using a dfs will prouve to find a solution faster. However in the case the fruit is close to the starting point, using a bfs will be a better option. This is because the dfs method use a stack so it is always the last node added that will be processed first. For the bfs method it uses a FIFO queue, reaching for its closest nodes first.

   (b) What are the advantages and disadvantages of using the tree and graph search **for this problem**. Which approach would you choose? **(2 pts)**

The advantages and disadvantages are whether they contain loops or not. With loops, trees can get stuck in the loop infinitely, while graphs avoid exploring the same states by keeping a set of explored states.

3. **Implement** a PacMan solver in Python 3. You shall extend the *Problem* class and implement the necessary methods -and other class(es) if necessary- allowing you to test the following four different approaches:

   - *depth-first tree-search (DFSt)*;
   - *breadth-first tree-search (BFSt)*;
   - *depth-first graph-search (DFSg)*;
   - *breadth-first graph-search (BFSg)*.

   Experiments must be realized (*not yet on INGInious!* use your own computer or one from the computer rooms) with the provided 10 instances. Report in a table the results on the 10 instances for depth-first and breadth-first strategies on both tree and graph search (4 settings above). Run each experiment for a maximum of 1 minute. You must report the time, the number of explored nodes as well as the number of remaining nodes in the queue to get a solution. **(4 pts)**

Inst.	BFS						DFS					
	Tree			Graph			Tree			Graph		
	T(s)	EN	RNQ	T(s)	EN	RNQ	T(s)	EN	RNQ	T(s)	EN	RNQ
i_01	0.01428	80	796	0.0011	7	72	0.0006	6	31	0.0044	6	31
i_02	0.0130	132	1096	0.0015	15	116	0.0002	3	18	0.0003	3	18
i_03	256.7447	677075	5060105	12.7485	79748	597326						
i_04	120.3330	290367	3204142	5.8571	24329	266037	0.0007	6	31	0.0007	6	31
i_05	1.2233	5841	54002	0.0964	621	5219	0.0008	6	36	0.0008	6	36
i_06	0.0210	235	1896	0.0019	26	208	0.0008	8	22	0.0004	8	22
i_07	0.1961	2184	16194	0.0202	262	1921	0.0003	6	21	0.0004	6	21
i_08	0.0021	57	282	0.0006	10	46	0.0002	8	8	0.0002	8	8
i_09	0.0067	68	526	0.0016	8	59	0.0007	6	32	0.0007	6	32
i_10	0.0136	132	1096	0.0014	15	116	0.0002	3	18	0.0002	3	18

T: Time — **EN**: Explored nodes — **RNQ**: Remaining nodes in the queue

4. **Submit** your program (encoded in **utf-8**) on INGInious. According to your experimentations, it must use the algorithm that leads to the best results. Your program must take as inputs the four numbers previously described separated by space character, and print to the standard output a solution to the problem satisfying the format described in Figure 3. Under INGInious (only 1 minute timeout per instance!), we expect you to solve at least 12 out of the 15 ones. **(6 pts)**

5. **Conclusion.**

   (a) How would you handle the case of some fruit that is poisonous and makes you lose? **(0.5 pt)**

In the same way as for a wall. Check whether the next move goes into the square of a poisoned fruit. Check in other directions if so, otherwise continue in the same direction.

   (e) Do you see any improvement directions for the best algorithm you chose? (Note that since we're still in uninformed search, *we're not talking about informed heuristics*). **(0.5 pt)**

A good improvement would be to use multiple bfs by spliting the grid, this would get rid of the problem of "how deep in our grid is the fruit".